

Uninformed Search

EECS 492

Introduction to Artificial
Intelligence

Today's Class

- **Search**
 - Preliminary definitions
 - A general search architecture
 - Breadth-first search
 - Uniform cost search
 - Depth-first search and variations

EECS 492 Topics

Boundaries are fuzzy!

A.I.

- Search
 - Uninformed
 - Informed
 - Local
 - Adversarial
- Constraint Satisfaction
- Logic
 - Propositional
 - First-Order
- Game theory

Machine Learning

- Decision Trees
- Linear Regression
- k-Nearest Neighbor
- SVMs
- Markov Decision Processes
- MCTS
- Reinforcement Learning

Neural Networks

- Feed forward (& deep)
- Convolutional
- Recurrent
- Transformers
- Generative

Topics subject to modest change

Lots of math across the entire course!

"Why not study only LLMs / modern GenAI / deep learning / neural networks?"

- Small models on personal machines, ok, but GenAI at scale is expensive
 - Cost to build yourself: millions to billions
 - Use another company's resources?
 - Significant energy / climate costs
- Neural networks are "black boxes", but in some domains, explainability is paramount.
- Neural networks are often improved when combined with other techniques.
- High quality research is also done in other areas of AI, and this trickles down to business & public sector applications.
- Neural network training is **search!**

7 trillion
worldwide
by 2030

If another technique can do the job, it will often be much cheaper and will require much less data.

Many techniques exist where the reason for a conclusion is more clear.

The field is constantly evolving. What's next?

NeurIPS 2024: Just a "few" examples

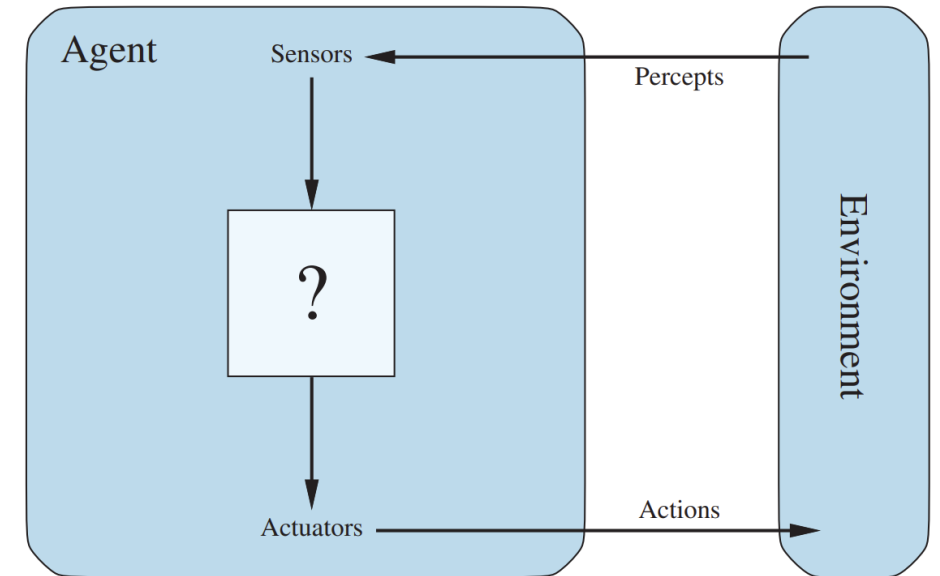
- [2024 proceedings](#)
- [All proceedings](#)

- SeeA*: Efficient Exploration-Enhanced **A* Search** by Selective Sampling
- Double-Ended Synthesis Planning with Goal-Constrained **Bidirectional Search**
- Minimizing UCB: a Better **Local Search** Strategy in Local Bayesian Optimization
- Hybrid Top-Down Global Causal Discovery with **Local Search** for Linear and Nonlinear Additive Noise Models
- **Monte Carlo Tree Search** based Space Transfer for Black Box Optimization
- Generating Code World Models with Large Language Models Guided by **Monte Carlo Tree Search**
- ReST-**MCTS***: LLM Self-Training via Process Reward Guided Tree Search
- Speculative **Monte-Carlo Tree Search**
- Transition Constrained **Bayesian** Optimization via **Markov Decision Processes**

And much more! See for yourself!

Back to Agents

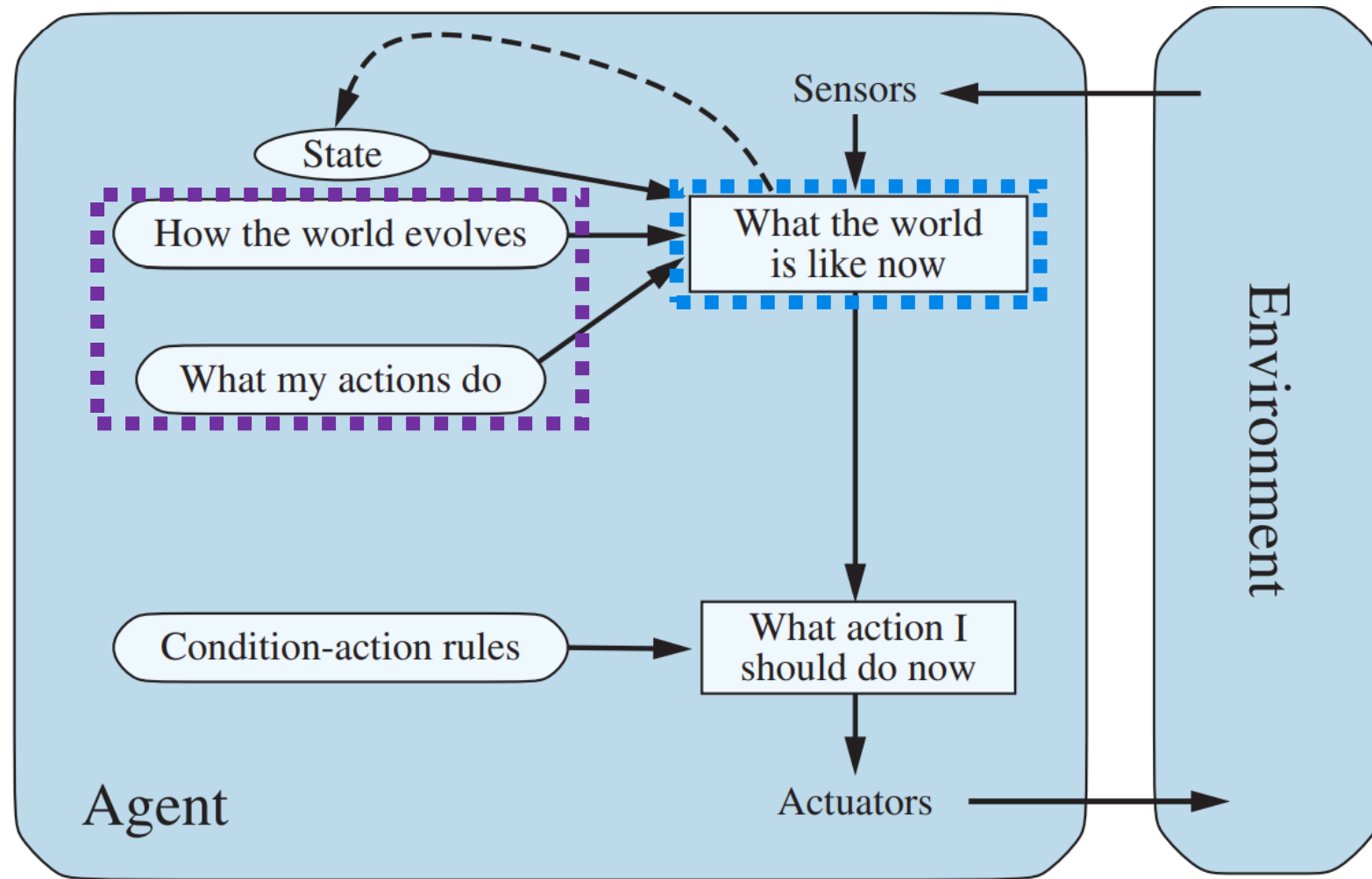
- An agent is anything that:
 - **Perceives** its environment through sensors
 - The things perceived are called *percepts*.
 - **Decides how to act** on its environment
 - Acts via *actuators*.
- An agent pursues a set of pre-specified goals, or a pre-specified strategy
 - Entrusted to act autonomously



Model-Based Reflex Agent

Also:

- Info about **how the world changes**.
- Maintain **internal belief** of the state of the world right now.



I just cleaned in a spot, so the floor is no longer dirty there.

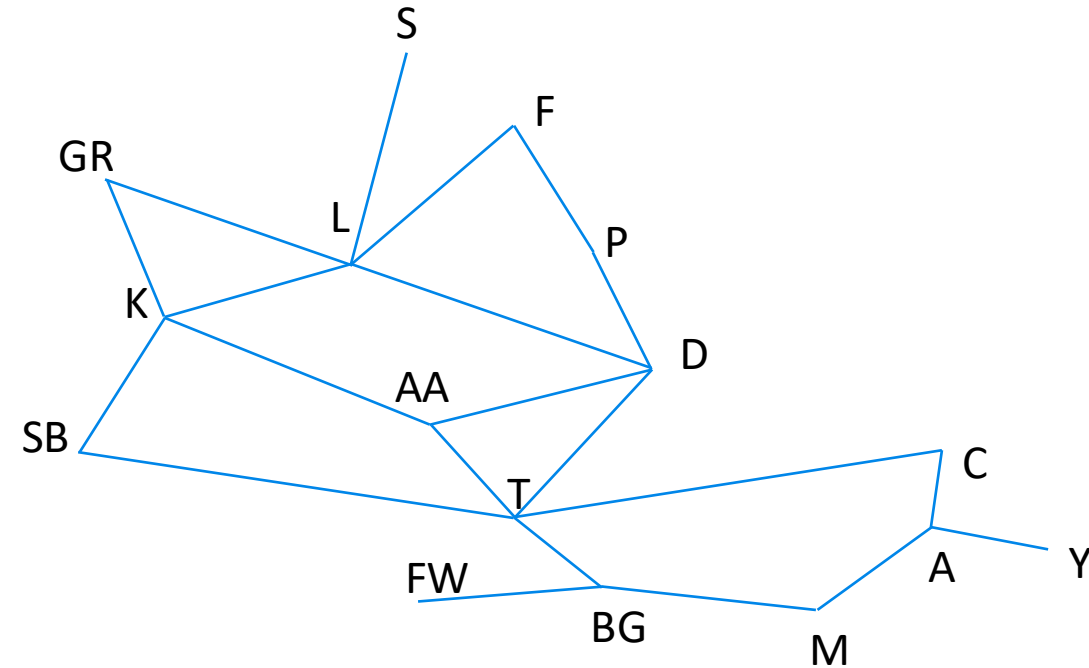
Preliminary Definitions

Problem Solving

- An agent needs a way to choose the action(s) to take
- Now:
 - *Searching* for the best action(s) to take
(for a model- and goal-based agent)
- **Real-world problems**
 - Route finding
 - VLSI layout (cell layout and channel routing)
 - Robotic manufacturing with partial ordering specified
 - Protein design (find sequence of amino acids that fold into a protein with desired properties)
- Let's start with an example, drawing general concepts from that

Find a route from AA to SB

- **Problem formulation:**
 - State: The city I'm in
 - Example: in AA
 - Action: Drive to an adjacent city (map is known)
 - Example: AA → K
 - Only certain cities are considered
 - Only certain roads between cities are considered
- **Initial state:** in AA
- **Goal state:** in SB
- **Search:**
 - Agent *simulates* various actions, trying to find sequences to reach the goal.
 - Example:
 - AA → T → SB
 - AA → D → P → F → L → GR → K → SB
 - ...
 - Agent picks a sequence to then *execute*.



Defining a search problem

1. State space –

- Set of possible states

Assume atomic.

2. Initial state

3. Goal state(s)

- Single goal?
- State with some characteristic?
(ex: any city with average yearly temperature in the 70's)

4. Actions

- $\text{Actions}(s)$ is a finite set of actions possible from state s

5. Transition model

- $\text{Result}(s, a)$ returns the state resulting from taking action a in state s

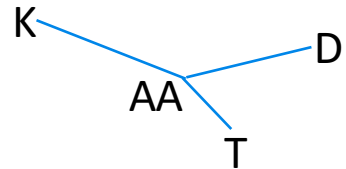
6. Action cost function

- $\text{Action_Cost}(s, a, s')$ or $c(s, a, s')$
- Assume positive for now.
- Cost of doing action a when in state s to get to state s' .

In our example:

1. $\{AA, SB, K, GR, L, S, F, P, D, T, FW, BG, M, A, C, Y\}$
2. AA
3. SB

4. $\text{Actions}(AA) = \{ToD, ToT, ToK\}$



5. $\text{Result}(AA, ToK) = K$
...

6. $\text{Action_Cost}(AA, ToK, K) = \dots$
(miles, or time, or tolls, or how ugly the scenery is, or ... any combination)

Defining a search problem

1. State space –

- Set of possible states

2. Initial state

3. Goal state(s)

- Single goal?
- State with some characteristic?
(ex: any city with average yearly temperature in the 70's)

4. Actions

- $\text{Actions}(s)$ is a finite set of actions possible from state s

5. Transition model

- $\text{Result}(s, a)$ returns the state resulting from taking action a in state s

6. Action cost function

- $\text{Action_Cost}(s, a, s')$ or $c(s, a, s')$
Cost of doing action a when in state s to get to state s' .

Definitions

- **path** – start state, end state, sequence of actions (or, when no ambiguity, a sequence of states)
- **path cost** – sum of *action costs* (*step costs*) for each action in a path
- **solution** – a path from initial state to goal state
- **optimal solution** – solution with lowest path cost

Environment properties:

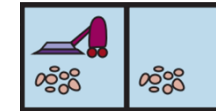
- Fully observable, single agent, deterministic, sequential, static, discrete, known

Defining a Search Problem: Vacuum

- Imagine just two possible locations: 
- Each may or may not have dirt. Initially, they're both dirty.
- Vacuum can be at either spot. Initially, it's at the left spot.
- Vacuum can move or clean at current location.

1. State space graph

2. Initial state –

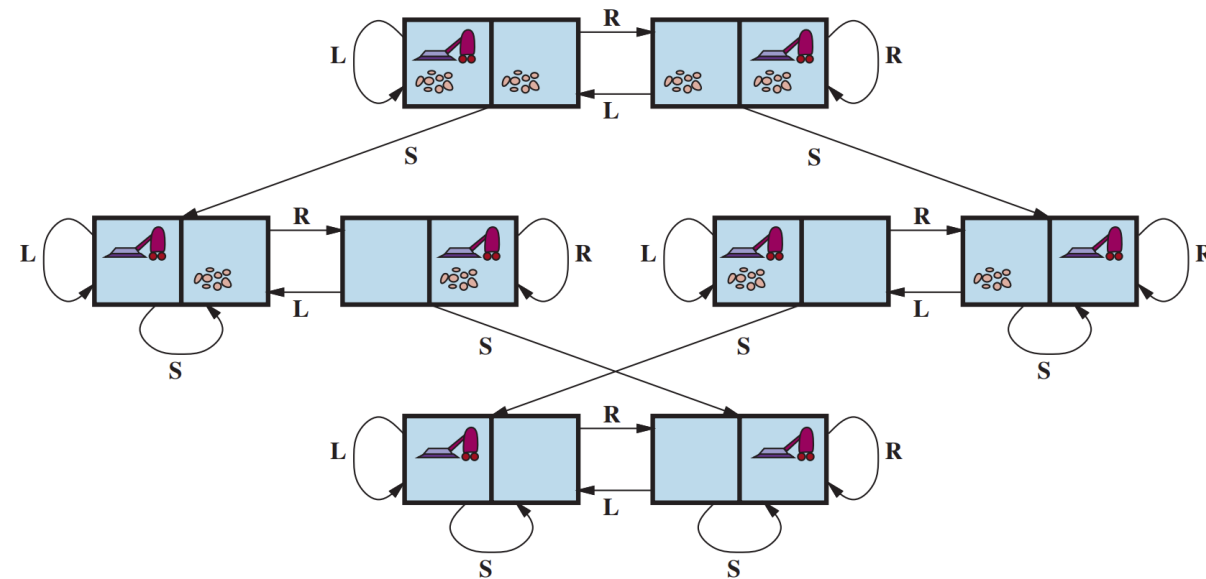


3. Goal state(s) – either state with both spots clean

4. Actions – L, R, S.
(maybe disallow invalid moves)

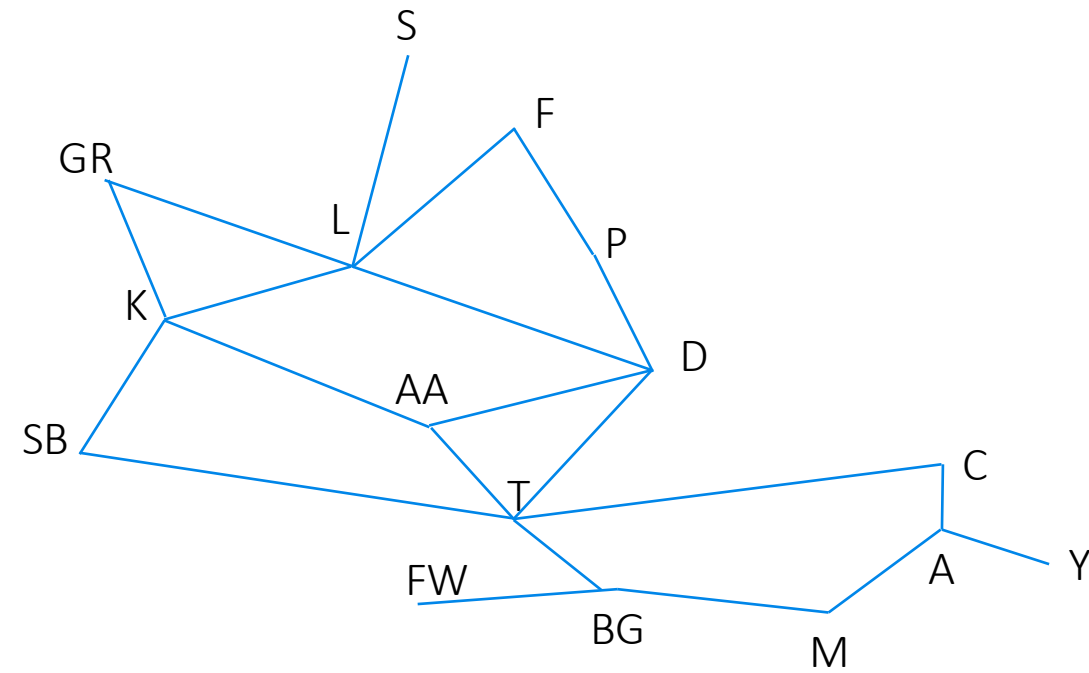
5. Transition model – Defined by the state space graph.

6. Action cost function – Let's suppose all actions are equal cost here.



Search Tree

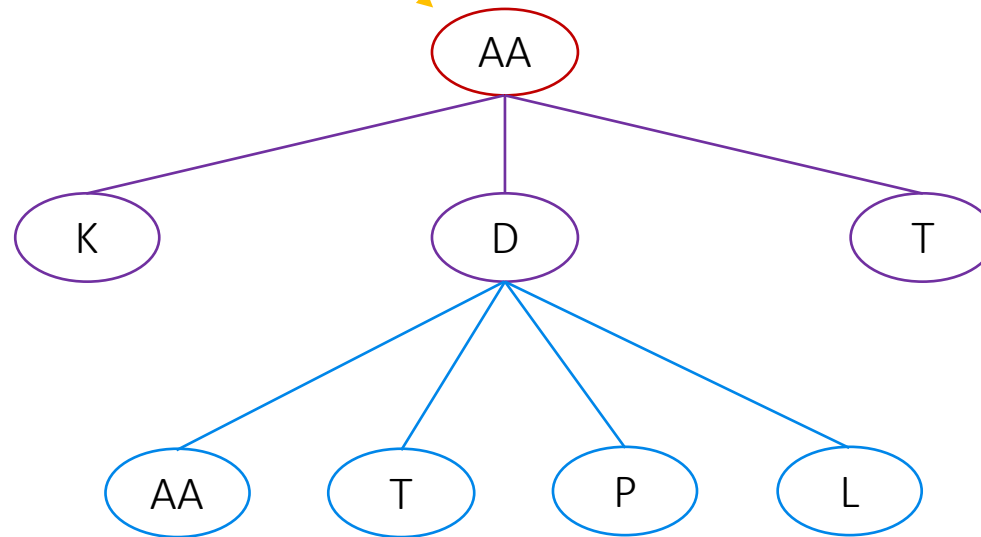
The search algorithms we'll consider for now all operate by creating a **search tree** on a state-space graph.



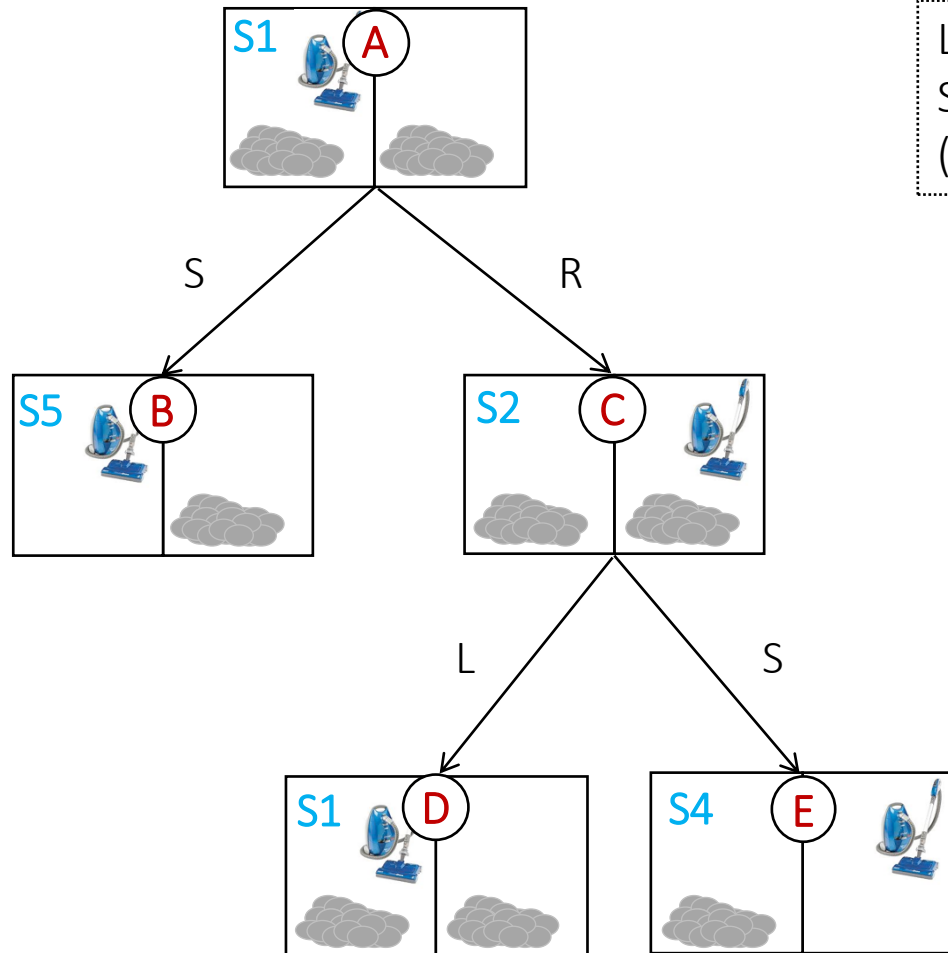
1) Initial state: **AA**

2) **Expand** AA (find all states reachable from AA with a single action)

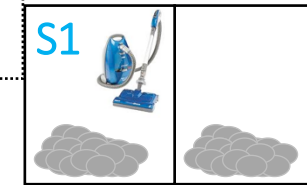
3) Select (somehow...) another node at any level (**K**, **AA**, **T**, **P**, **L**, or **T**), and expand it...



Vacuum: Search Tree



Let's label each **node** (**A** - **E**).
Some **nodes** contain the same **state**!
(**A**) and (**D**) here both have state **S1**.)



Parent node also contains *its* parent node (the grandparent), and so on, to the root.

So each node has information about entire path from root (initial state).

To reduce notation, I won't always *show* this, though...

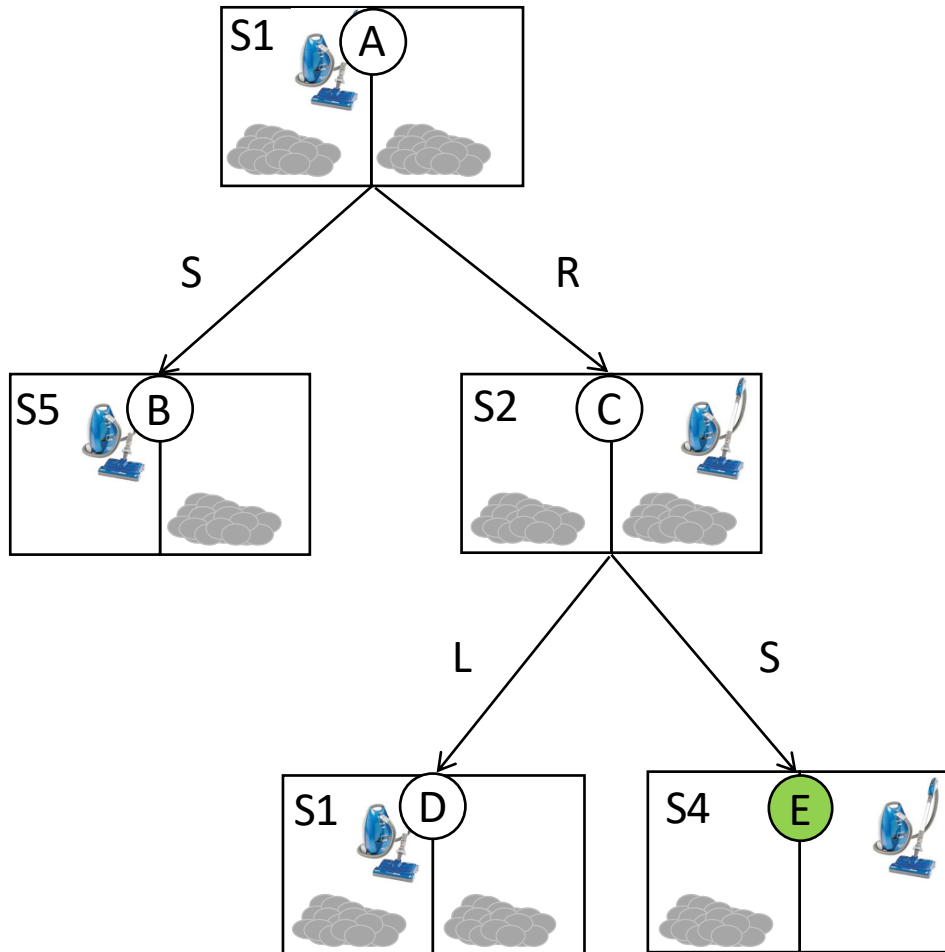
State – the way the world is
vs.
Node – part of the search tree

State and Node aren't the same thing!

- A **Node** contains:
 - **State**
 - Parent **node**
 - Action to get from parent to this node
 - path cost $g(n)$ – cost from root node to this node

Omitting the non-move loops here, for brevity.

Vacuum: Node Example



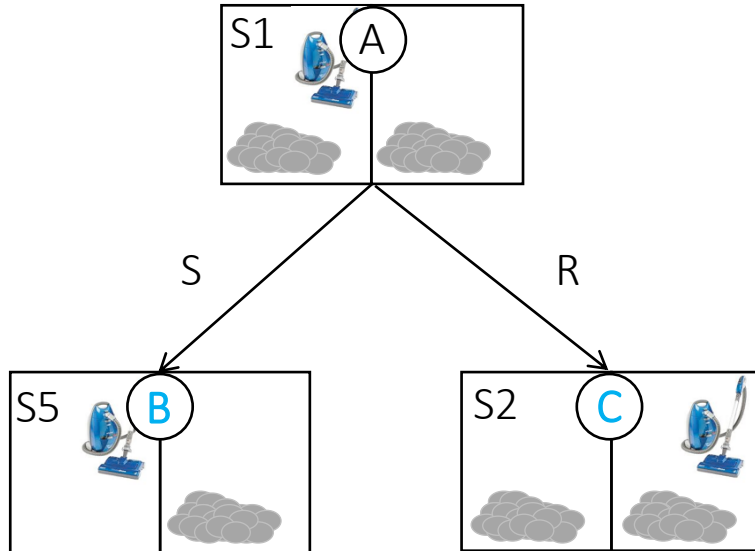
Let's assume each action (S, L, R) has the same cost. We'll just call that cost 1.

- Node **E** contains:
 - State **S4**
 - Parent **C**
 - Action **S**
 - $g(n) = 2$

Uniformed Search

- A Node contains:
 - State
 - Parent node
 - Action to get from parent to this node
 - path cost $g(n)$ – cost from root node to this node

Vacuum: Vocabulary

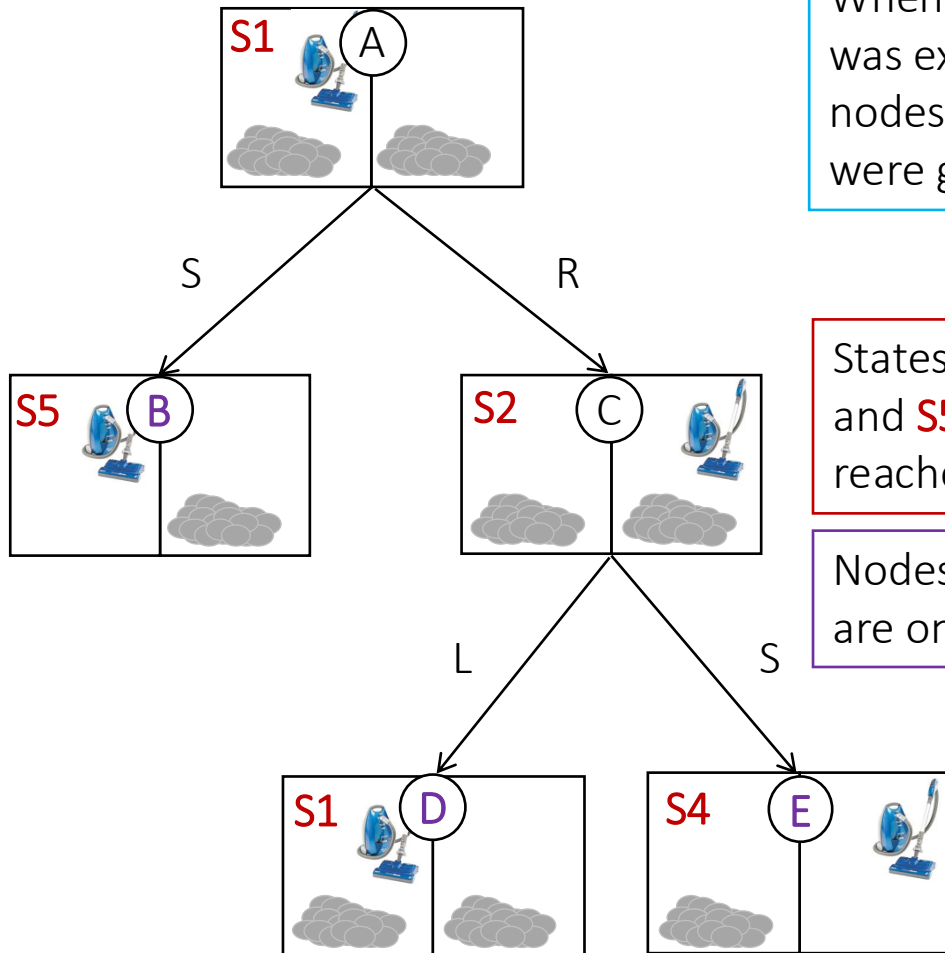


When node A was expanded, nodes B and C were generated.

- *expand* a node:
 - determine applicable actions
 - determine resulting states (*successors*)
 - *generate* nodes for each successor and add them to the search tree

- A Node contains:
 - State
 - Parent node
 - Action to get from parent to this node
 - path cost $g(n)$ – cost from root node to this node

Vacuum: Vocabulary



When node A was expanded, nodes B and C were generated.

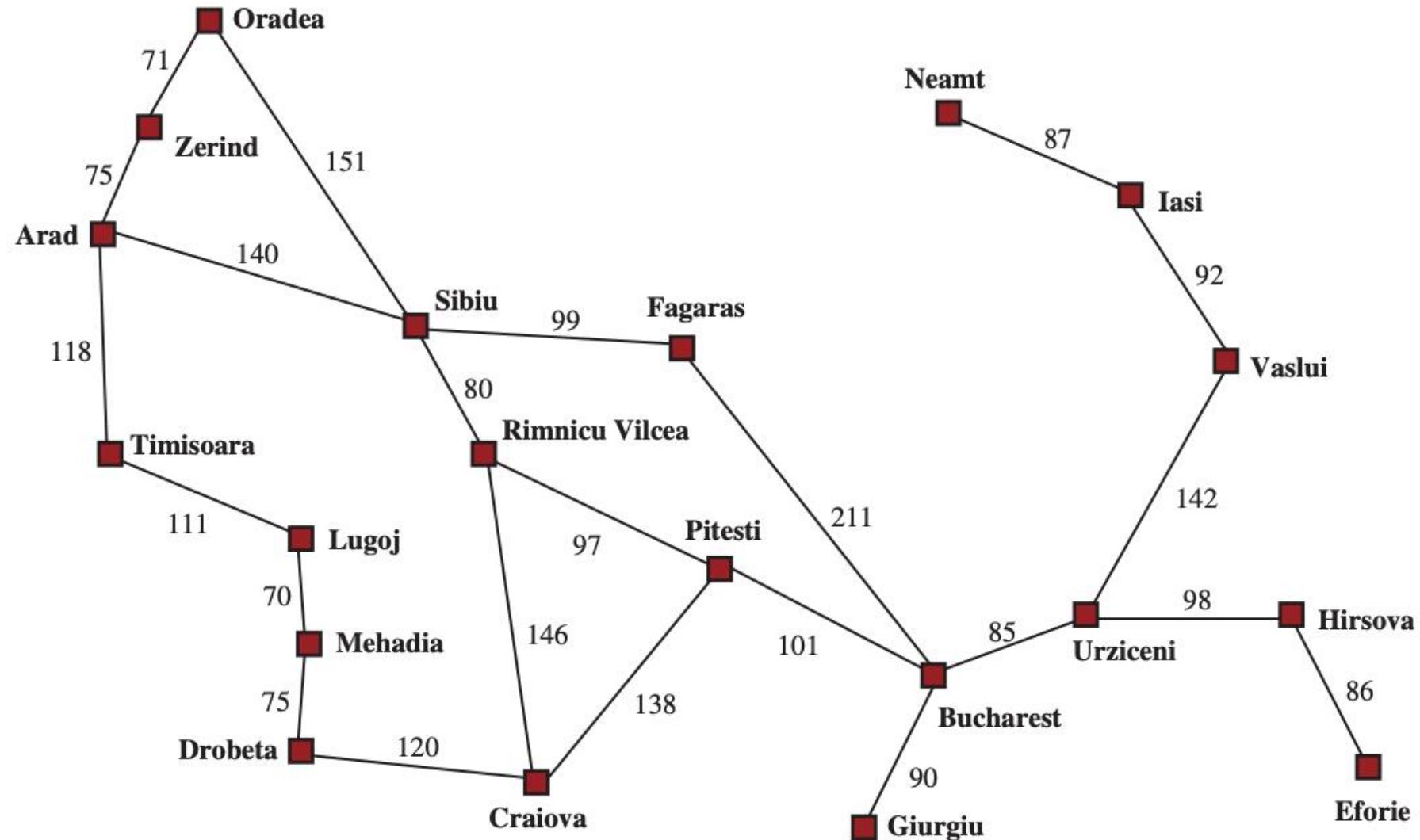
States S1, S2, S4, and S5 have been reached.

Nodes B, D, and E are on the frontier.

- *expand* a node:
 - determine applicable actions
 - determine resulting states (*successors*)
 - *generate* nodes for each successor and add them to the search tree
- A *state* has been *reached* if at least one node has been *generated* that contains it.
- A *node* is on the *frontier* if it has not been expanded yet. It's a leaf that hasn't been expanded (as opposed to a leaf that we did try to expand but that had no children).

- A Node contains:
 - State
 - Parent node
 - Action to get from parent to this node
 - path cost $g(n)$ – cost from root node to this node

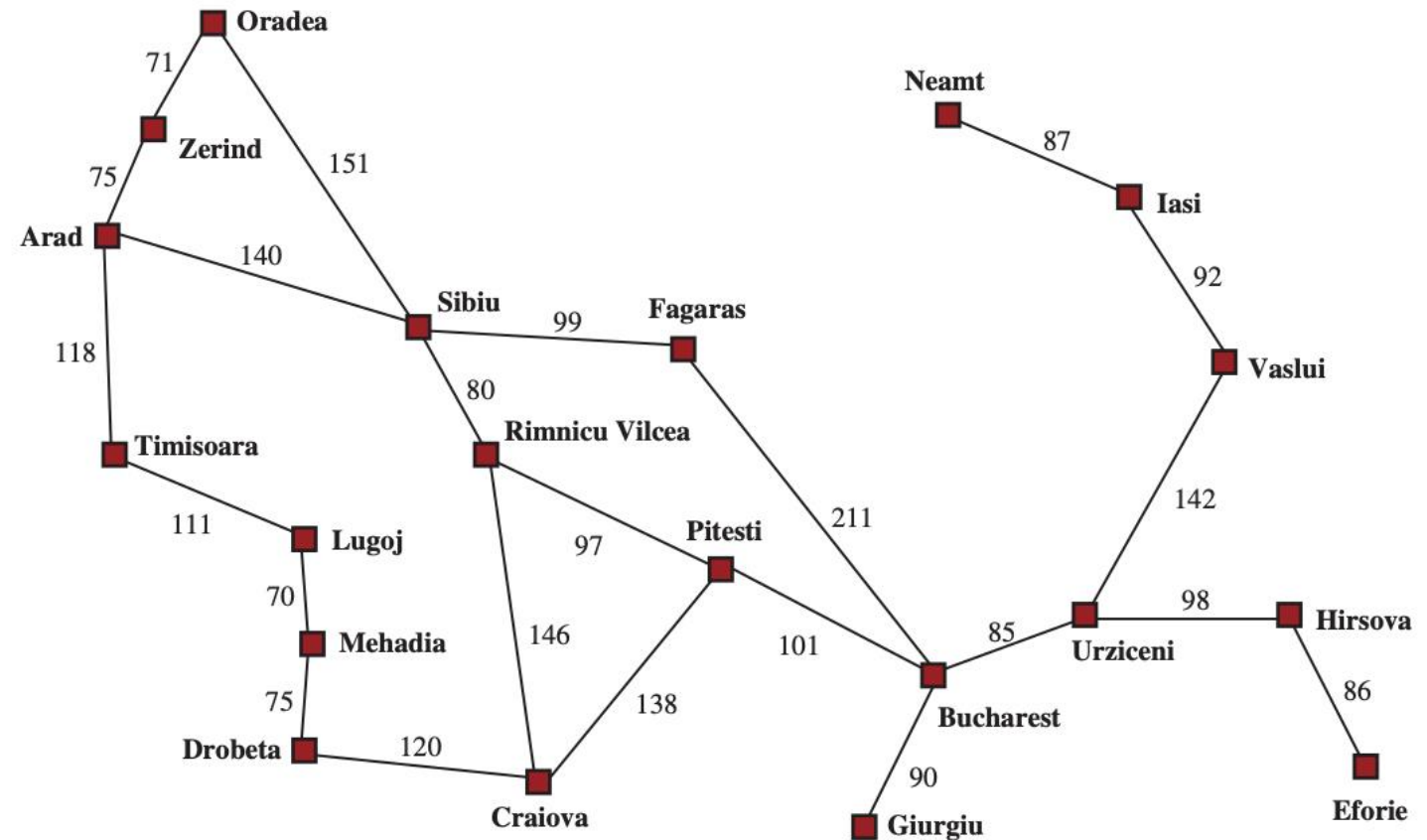
Exercise: Toy Map of Romania (distances in miles)





Exercise

- Let's say we're trying to get from Arad to Oradea
- What are the following quantities?
 - INITIAL-STATE
 - ACTIONS(Arad)
 - RESULT(Arad, Arad->Sibiu)
 - IS-GOAL(Sibiu)
 - ACTION-COST(Arad, Arad->Sibiu, Sibiu)



Measuring Performance

- Completeness
 - Is the algorithm guaranteed to find a solution when there is one?
- Optimality
 - Does the strategy find the optimal solution?
 - Can define in terms of path cost
- Time complexity
 - How long does it take to find a solution?
- Space complexity
 - How much memory is needed to perform the search?

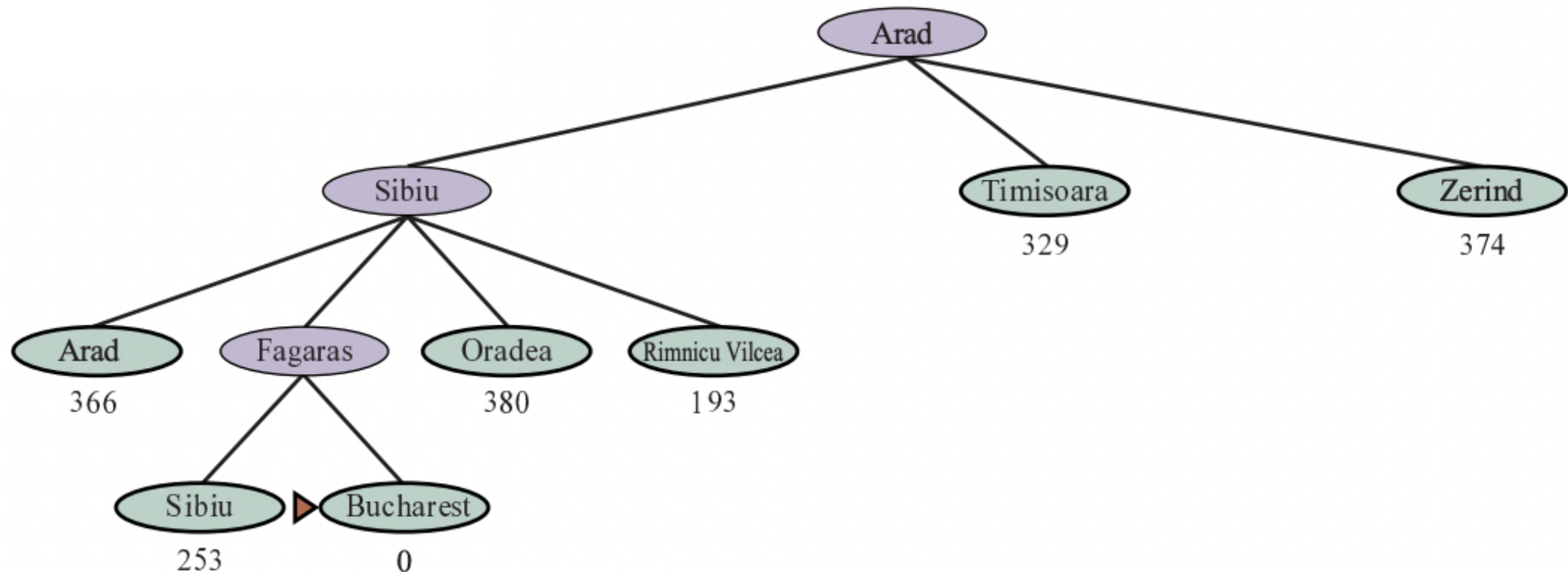
Graph Properties

- **Branching factor (b)** = maximum number of successors for any node
- **Depth (d)** = shallowest goal node (number of actions in an optimal solution)
- **Maximum path length (m)** = longest length of any path in the search ~~space~~ tree (max number of actions in any path)



Exercise

Let's say we generate this search tree, searching from Arad to Bucharest. What is the branching factor (b), depth (d), and maximum path length (m) for this tree?

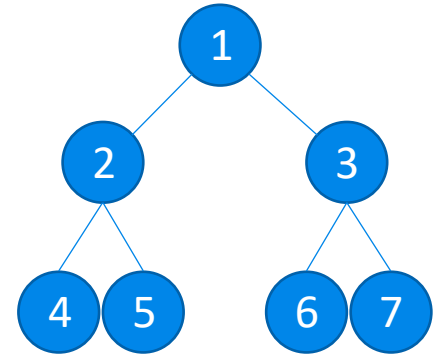


A General Search Architecture

Let's think about a general search architecture

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

The problem:



Frontier: []

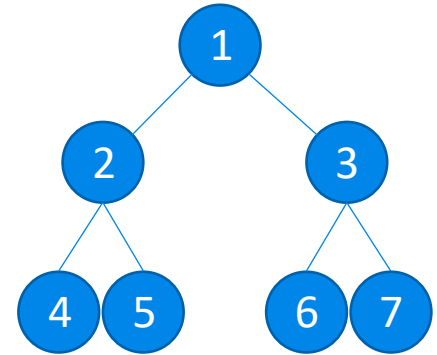
Reached: []

Let's think about a general search architecture

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

f is an evaluation function, we'll talk about this at length

The problem:



Frontier: []

Reached: []

Let's think about a general search architecture

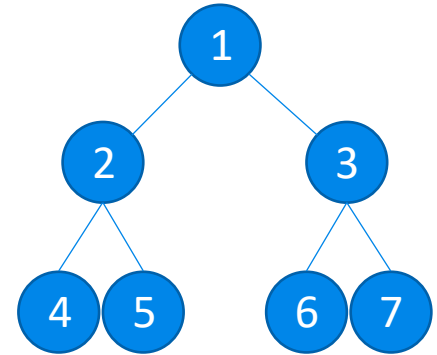
function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

The problem:



Frontier: [1]

Reached: [1]

Let's think about a general search architecture

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

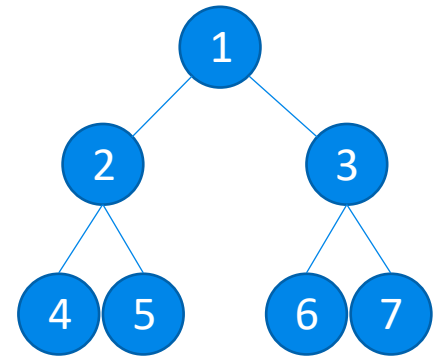
node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

The problem:



Frontier: [1]

Reached: [1]

Let's think about a general search architecture

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

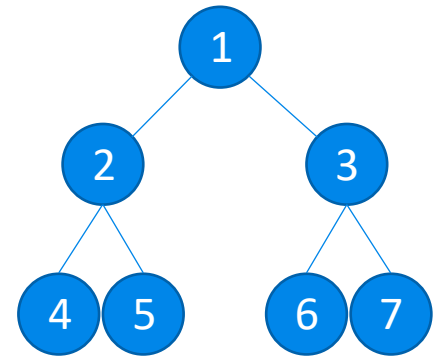
frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

The problem:



Frontier: [1]

Reached: [1]

We are now considering node 1

Let's think about a general search architecture

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

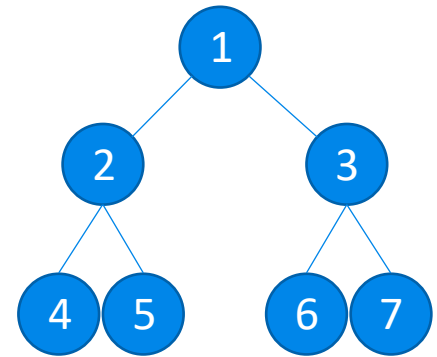
frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

The problem:



Frontier: []

Reached: [1]

We are now considering node 1

Let's think about a general search architecture

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

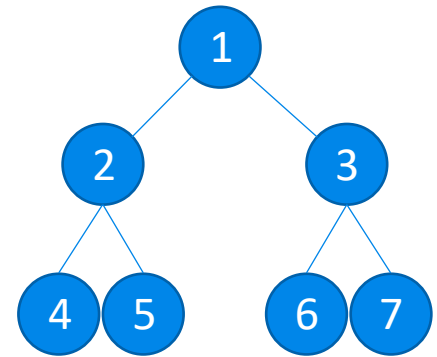
reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

The problem:



Frontier: []

Reached: [1]

Node 1 isn't a goal node

Let's think about a general search architecture

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

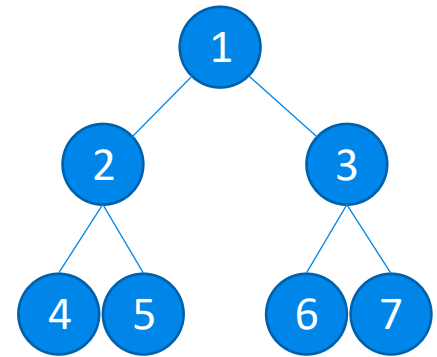
while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

The problem:



Frontier: []

Reached: [1]

So, we'll expand it (find its children)

Let's think about a general search architecture

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

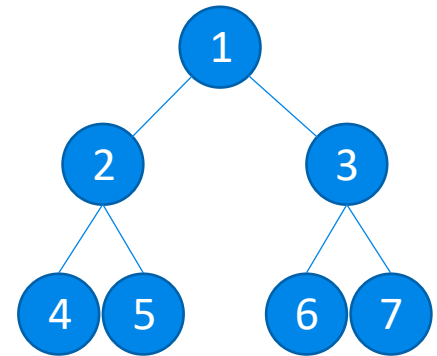
node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

The problem:



Frontier: []

Reached: [1]

So, we'll expand it (find its children) – they are 2 and 3, we'll consider node 2

Let's think about a general search architecture

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

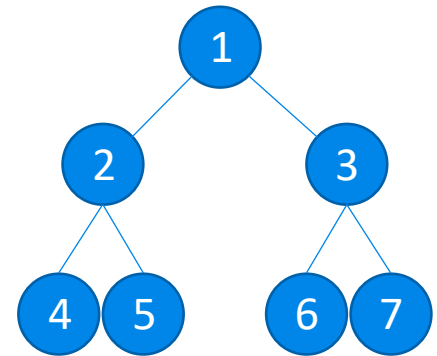
node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

The problem:



Frontier: []

Reached: [1]

So, *s* is the state of node 2

Let's think about a general search architecture

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

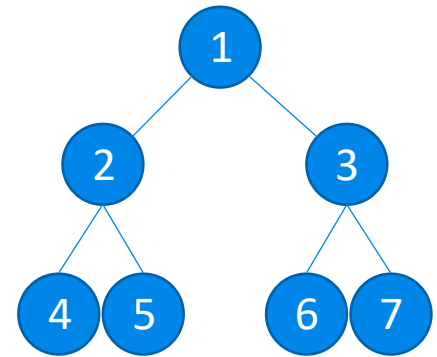
if *s* is not in *reached* **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**

reached[*s*] $\leftarrow$ *child*

add child to frontier

- If state *s* **not in** frontier, **add** child to frontier
- Else, **update** the node currently in frontier with the child node

The problem:



Frontier: [2]

Reached: [1, 2]

Let's think about a general search architecture

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

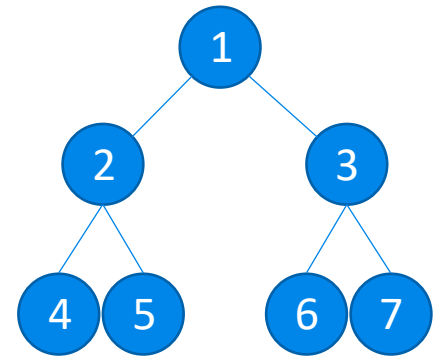
s $\leftarrow$ *child*.STATE

if *s* is not in *reached* **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**

reached[*s*] $\leftarrow$ *child*

 add *child* to *frontier*

The problem:



Frontier: [2, 3]

Reached: [1, 2, 3]

Then 3, then onwards...

Let's think about a general search architecture

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

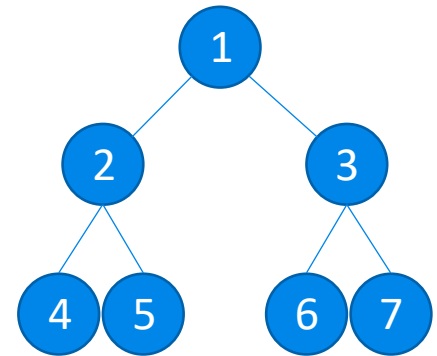
if *s* is not in *reached* **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**

reached[*s*] $\leftarrow$ *child*

 add *child* to *frontier*

return *failure*

The problem:



Frontier: []

Reached: [...]

What does it mean to expand?

function EXPAND(*problem, node*) **yields** nodes

$s \leftarrow node.STATE$

for each *action* **in** *problem.ACTIONS(s)* **do**

$s' \leftarrow problem.RESULT(s, action)$

$cost \leftarrow node.PATH-COST + problem.ACTION-COST(s, action, s')$

yield NODE(STATE= s' , PARENT=*node*, ACTION=*action*, PATH-COST= $cost$)

What does it mean to expand?

function EXPAND(*problem, node*) **yields** nodes

$s \leftarrow node.STATE$  Your current state

for each *action* **in** *problem.ACTIONS(s)* **do**

$s' \leftarrow problem.RESULT(s, action)$

$cost \leftarrow node.PATH-COST + problem.ACTION-COST(s, action, s')$

yield NODE(STATE= s' , PARENT=*node*, ACTION=*action*, PATH-COST= $cost$)

What does it mean to expand?

function EXPAND(*problem*, *node*) **yields** nodes

$s \leftarrow \text{node.STATE}$  Your current state

for each *action* **in** *problem.ACTIONS*(*s*) **do**  All the things you could do in the state

$s' \leftarrow \text{problem.RESULT}(s, \text{action})$

$\text{cost} \leftarrow \text{node.PATH-COST} + \text{problem.ACTION-COST}(s, \text{action}, s')$

yield NODE(STATE= s' , PARENT=*node*, ACTION=*action*, PATH-COST=*cost*)

What does it mean to expand?

function EXPAND(*problem*, *node*) **yields** nodes

$s \leftarrow \text{node.STATE}$  Your current state

for each *action* **in** *problem.ACTIONS*(*s*) **do**  All the things you could do in the state

Where you
could end up

 $s' \leftarrow \text{problem.RESULT}(s, \text{action})$

$\text{cost} \leftarrow \text{node.PATH-COST} + \text{problem.ACTION-COST}(s, \text{action}, s')$

yield NODE(STATE= s' , PARENT=*node*, ACTION=*action*, PATH-COST=*cost*)

What does it mean to expand?

function EXPAND(*problem*, *node*) **yields** nodes

$s \leftarrow \text{node.STATE}$  Your current state

for each *action* **in** *problem.ACTIONS(s)* **do**  All the things you could do in the state

Where you
could end up

 $s' \leftarrow \text{problem.RESULT}(s, \text{action})$

$\text{cost} \leftarrow \text{node.PATH-COST} + \text{problem.ACTION-COST}(s, \text{action}, s')$

yield NODE(STATE= s' , PARENT=*node*, ACTION=*action*, PATH-COST=*cost*)

How expensive it
would be to get
there



What does it mean to expand?

function EXPAND(*problem*, *node*) **yields** nodes

$s \leftarrow \text{node.STATE}$  Your current state

for each *action* **in** *problem.ACTIONS*(*s*) **do**  All the things you could do in the state

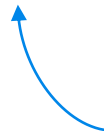
Where you
could end up

$s' \leftarrow \text{problem.RESULT}(s, \text{action})$ 

$\text{cost} \leftarrow \text{node.PATH-COST} + \text{problem.ACTION-COST}(s, \text{action}, s')$

yield NODE(STATE=*s'*, PARENT=*node*, ACTION=*action*, PATH-COST=*cost*)

How expensive it
would be to get
there

 generates a sequence of values, each time yield is encountered

Uninformed Search

A quick review of BFS, Dijkstra, and DFS

- We're going to go quickly through EECS 281 topics
- For a more detailed review of them, please see the slides of this week's discussion section
- Helpful document to compare all search algorithms we'll look at:
 - [Canvas > Files > Lecture Slides > Search Algorithms at a Glance.pdf](#)

Uninformed Search

- Information available to algorithm
 - Only what is provided by problem definition
 - No information about how close a state is to the goal state

BFS in the general search architecture

function BREADTH-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a first-in-first-out queue, with *node* as an element*

reached $\leftarrow$ a set of states, with one entry with value *node*.STATE

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

for each *child* **in** EXPAND(*problem*, *node*) **do**

if *problem*.IS-GOAL(*child*.STATE) **then return** *child*

s $\leftarrow$ *child*.STATE

if *s* is not in *reached* **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**

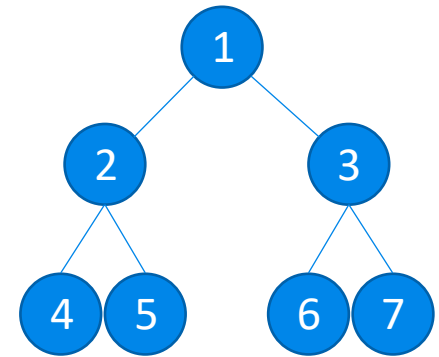
reached[*s*] $\leftarrow$ *child*

add child to frontier

return *failure*

Early **goal test** – check if a node is a solution as soon as it is *generated*, rather than waiting until it is popped off the queue.

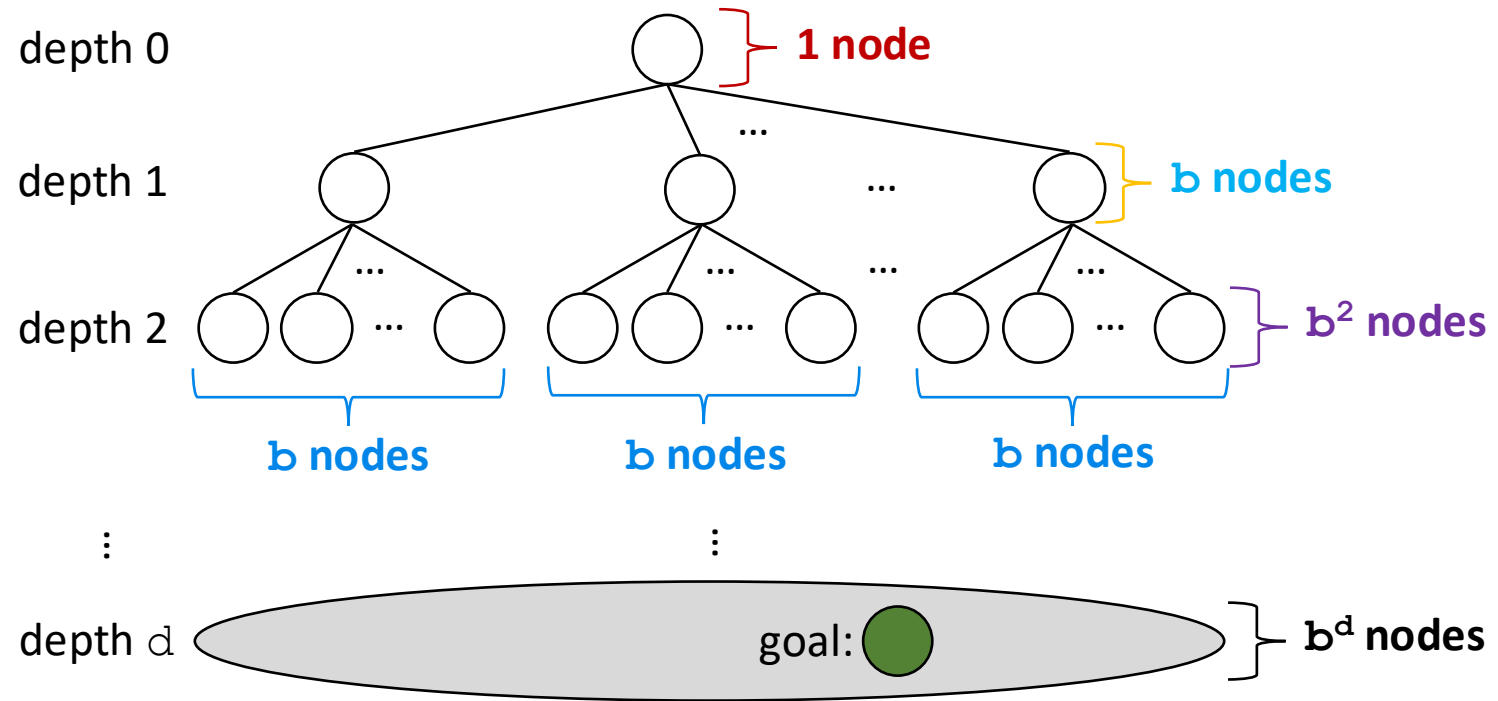
The problem:



Reached: stores **states**
Frontier: **first-in-first-out** queue

BFS: Time Complexity

- Assumptions:
 - We have a uniform branching factor, b
 - The depth of the shallowest solution is d
- BFS generates complete search trees at depth $\leq d$
- What if BFS checked for goal state when expanding nodes?



Worst case number of nodes generated:

$$1 + b + b^2 + \dots + b^d = O(b^{d+1})$$

Thanks to **early goal test**, the moment we generate the goal , we return.
If late goal test, then space is $O(b^{d+1})$ because $d+1$ nodes will be entered to frontier

BFS Space Complexity

- Follows the analysis in the previous slide
- BFS stores all nodes (frontier)
 - Frontier $\rightarrow O(b^d)$
- Space complexity is $O(b^d)$ thanks to early goal testing
- Generally, BFS runs out of space before it runs out of time!

BFS: Search Performance Criteria

- Is BFS optimal?
 - BFS always find solution with minimal number of actions, because when it is generating nodes at depth d , it has already generated all the nodes at depth $d-1$
 - If all the actions have the same cost, **it is optimal!**
 - If all the actions don't have the same cost, **it may not be optimal**
- Is BFS complete?
 - Yes – it systematically searches through all possible paths at each depth

BFS Limitations

- Time complexity is a problem
- Space complexity is the killer!

Depth	Nodes	Time	Memory
2	110	0.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

- Rule of thumb: exponential-complexity search problems **cannot** be solved by uninformed methods for any but the smallest instances!

Uniform-Cost Search (Dijkstra's algorithm)

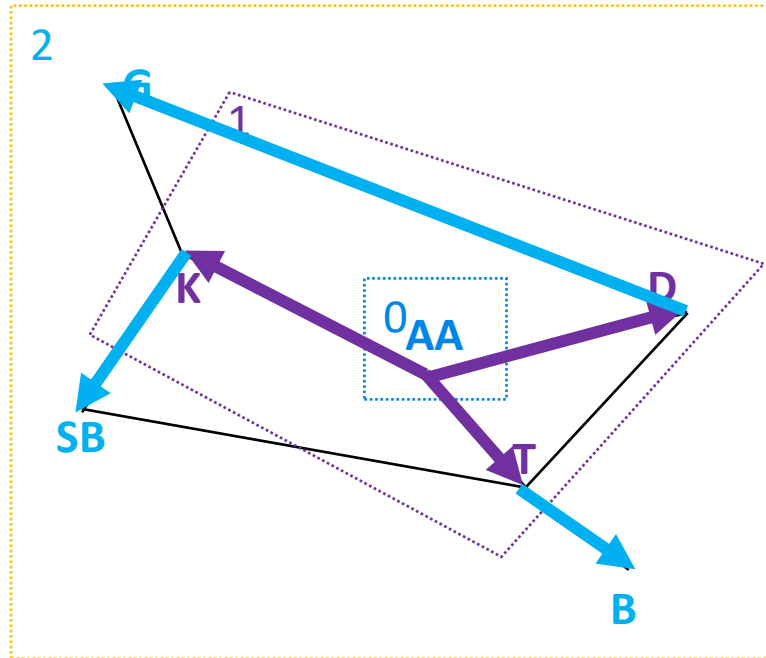
- Remember, BFS is optimal if all actions have the same cost
- Let's extend the algorithm so that it is optimal even if all actions do not have the same cost!
- Uniform-Cost Search
 - Expands node n with the lowest path cost $g(n)$
 - Frontier is stored in a priority queue ordered by g

BFS vs. UCS

Let's consider an image that is scaled by path cost.

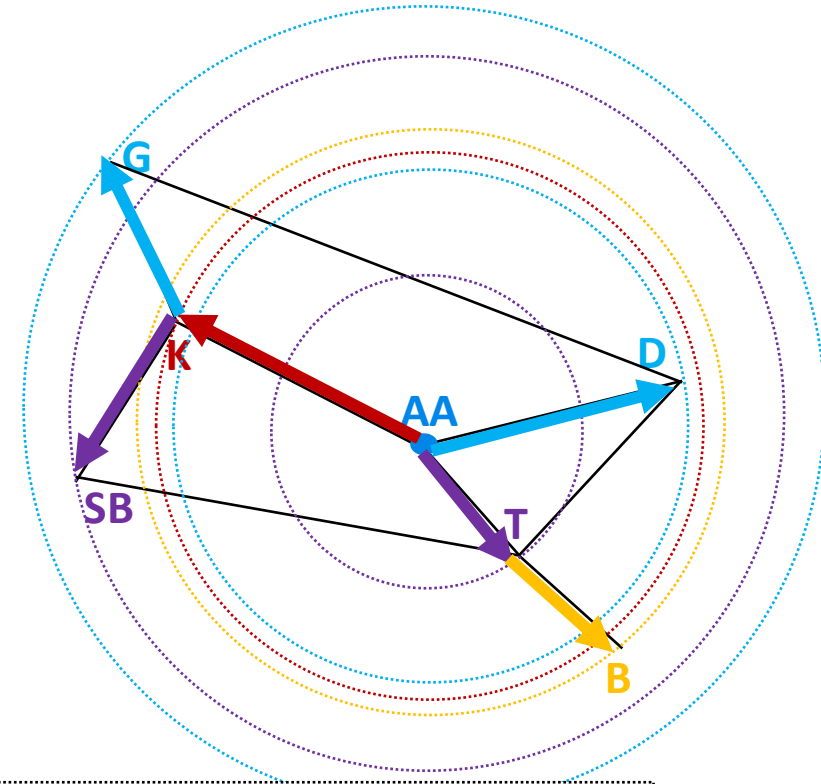
BFS

Expand nodes in FIFO manner



UCS

Expand node on frontier with lowest path cost



UCS uses a priority queue!

UCS in the general search architecture

function UNIFORM-COST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a *priority queue ordered by *f**, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

if *s* is not in *reached* **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**

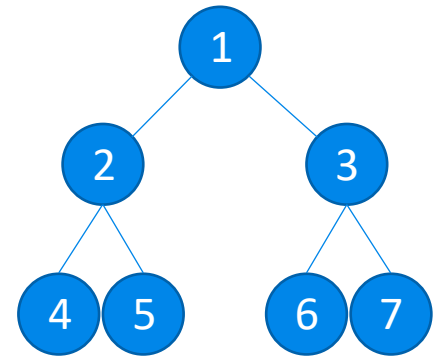
reached[*s*] $\leftarrow$ *child*

add child to frontier

return *failure*

Late goal test – checking whether a node is a solution node when it is popped off the queue

The problem:

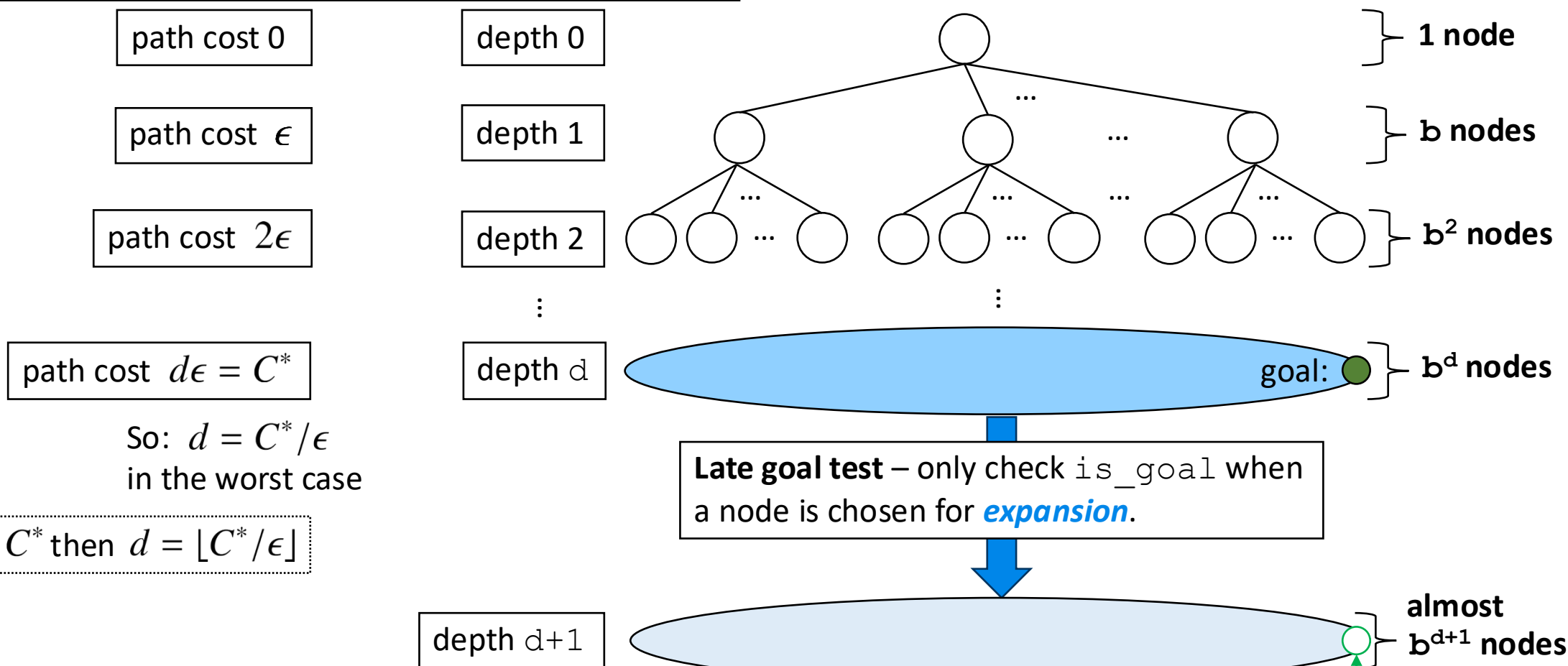


Reached: stores *nodes*
Frontier: *priority* queue, path-cost

Time and Space Complexity

- Worst case, each step has minimum cost ϵ .

C^* = the cost of the optimal solution
 ϵ = the minimum step-cost



Worst case number of nodes generated:

$$\left(\sum_{i=0}^{d+1} b^i \right) - gs = O(b^{d+1}) = O(b^{\lfloor C^*/\epsilon \rfloor + 1})$$

UCS: Search Performance Criteria

- Is UCS Optimal?
 - Yes! When node n is expanded, the optimal path to n has been found
 - (Assuming step-costs are non-negative)
- Is UCS Complete?
 - Yes, assuming all step costs are greater than a small constant, epsilon ϵ
- Time and Space Complexity
 - C^* = the cost of the optimal solution
 - ϵ = the minimum step-cost
 - $O(b^{1+\lceil C^*/\epsilon \rceil})$

DFS in the general search architecture

Note: We assume there are no cycles and do not save a *reached* set (saves memory!)

function DEPTH-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a *last-in first-out* queue, with *node* as an element

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

Late goal test – checking whether a node is a solution node when it is popped off the queue

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

add child to frontier

return *failure*

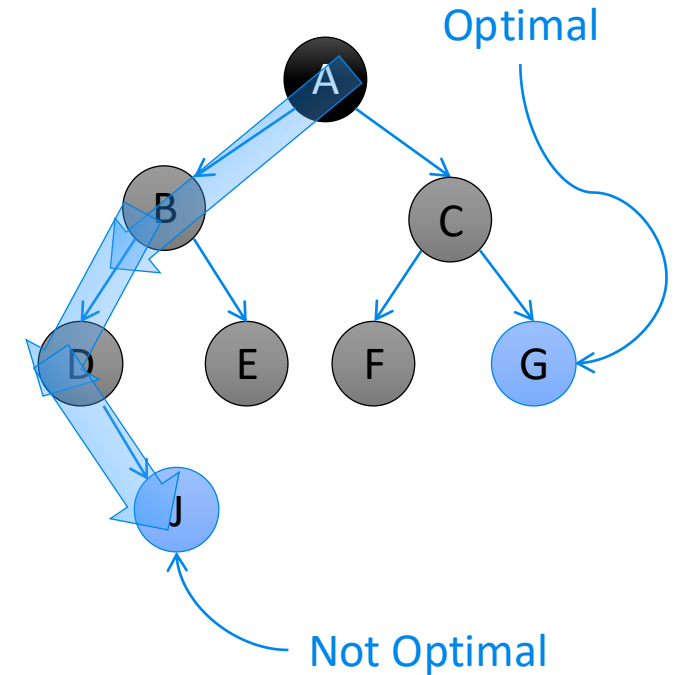
Reached: not necessary
Frontier: *last-in-first-out* queue

DFS saves memory

- Why?
 - No reached structure
 - DFS often applied to problems where cycles are not present or rare
- What if there are cycles?
 - Follow chain of parent pointers up the tree to see if state at the end of the path has appeared earlier

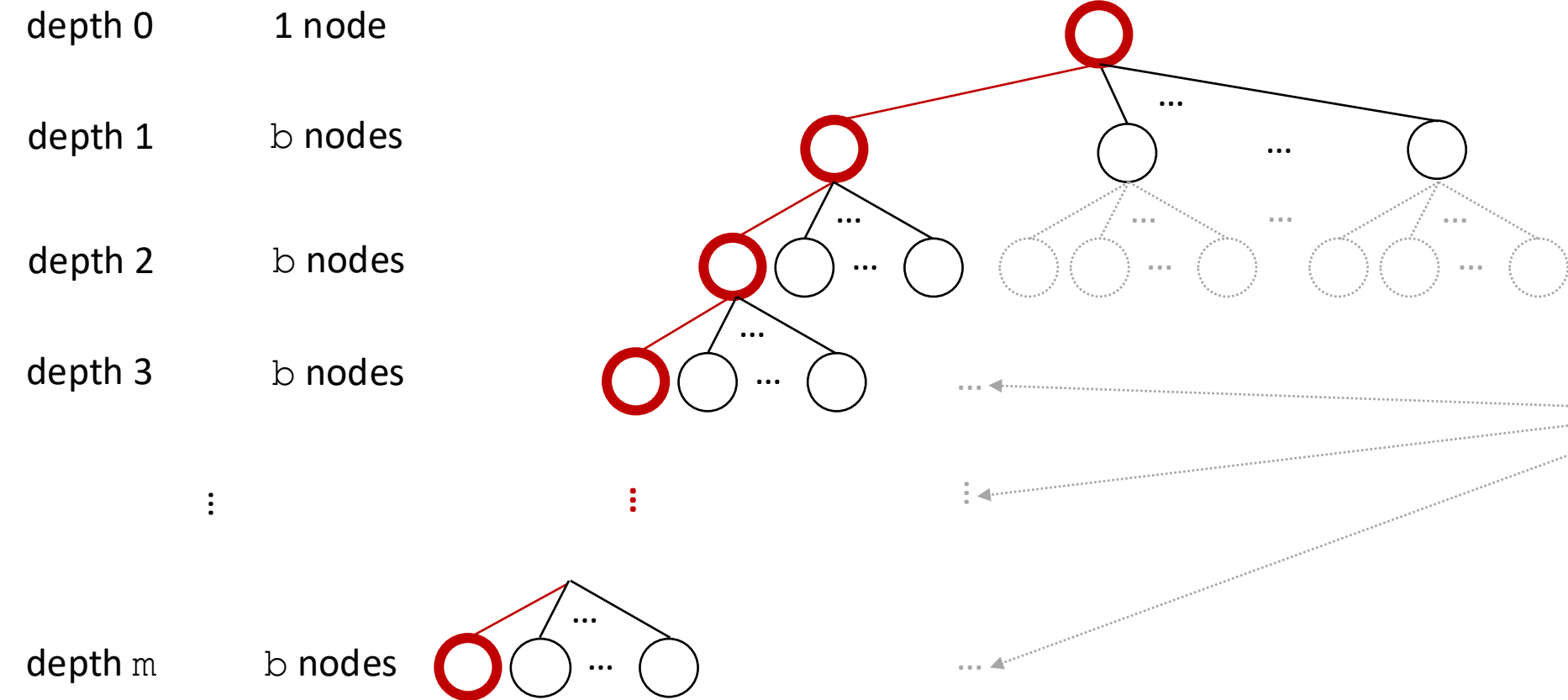
DFS: Search Performance Criteria

- Is DFS Optimal?
 - No - returns the first solution it finds, even if it's not the cheapest
- Is DFS Complete? It depends...
 - **Complete**: Graph search (finite spaces)
 - **NOT Complete**: Tree search (finite spaces)
 - **NOT Complete**: Both versions (infinite spaces)



DFS Space Complexity

Let m = maximum depth of the **tree**.
Maybe much bigger than $d!$ (depth of sol)



Nodes that won't be reached until later (not generated yet).

Worst case, number of nodes in memory at one time:

$$1 + \underbrace{b + b + \dots + b}_{m \text{ of these}} = O(bm)$$

Uniformed Search

No maximum depth m ? This is the case in infinite spaces
Then DFS is not complete and could use unbounded space.

DFS Performance

Space complexity

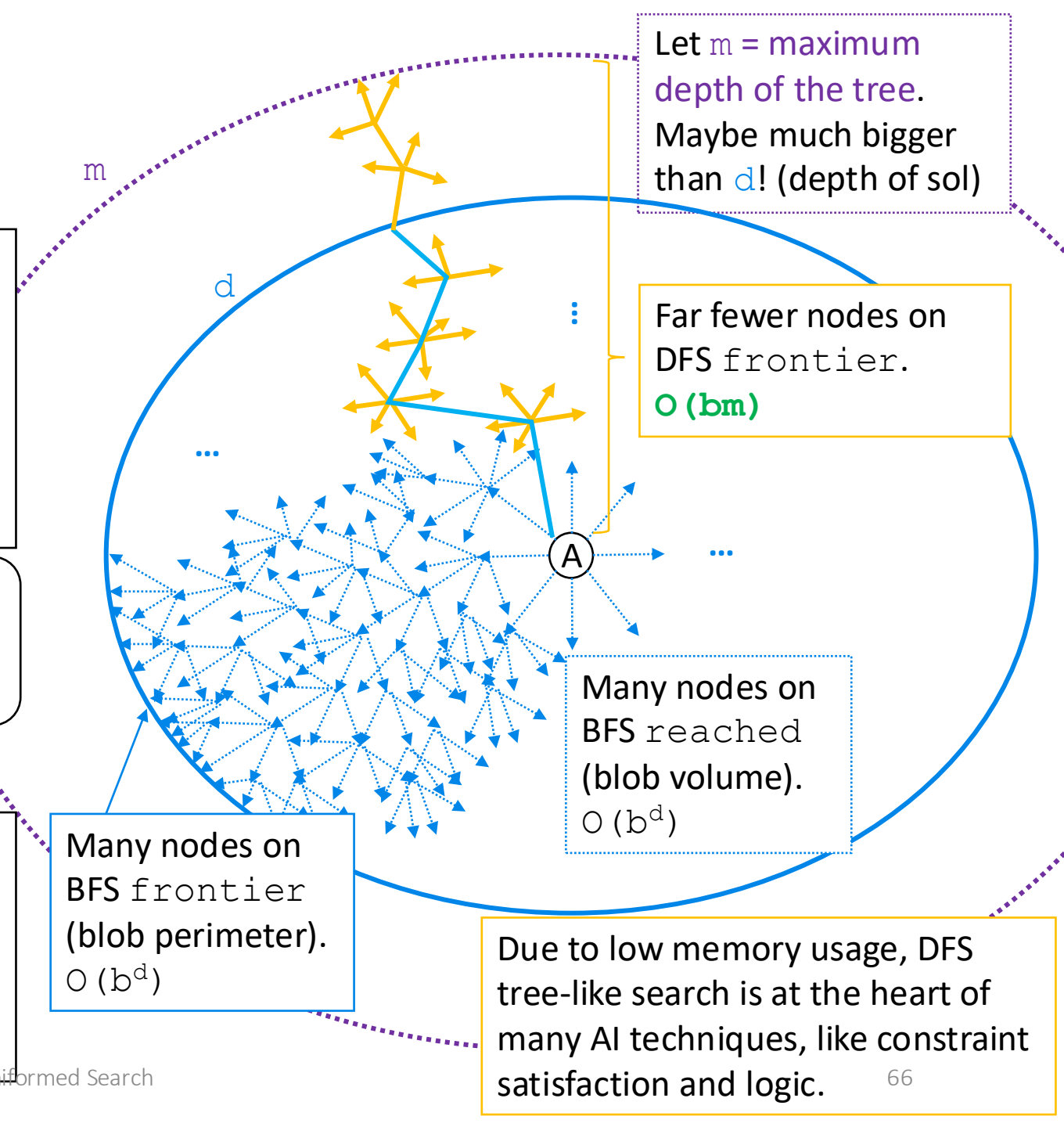
- When **tree-like search** is feasible, DFS requires **far less memory** than BFS or UCS:
 - No reached
 - frontier is much smaller

States in mem. for $b = 10$, $d = 16$, $m = 100$:

- BFS: $O(b^d)$ 10,000,000,000,000,000
- DFS: $O(bm)$ 1,000 ← **Huge Savings!**

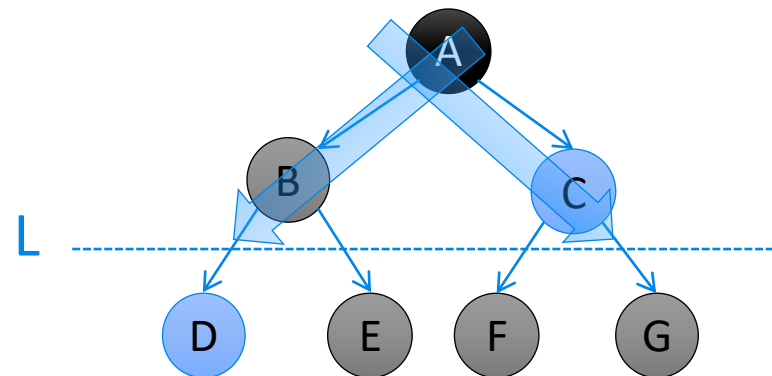
Time complexity

- $O(b^m)$ – still may have to search the entire space to the maximum depth in the tree. (Still have to "fill up the blob", just not *saving* those nodes.)



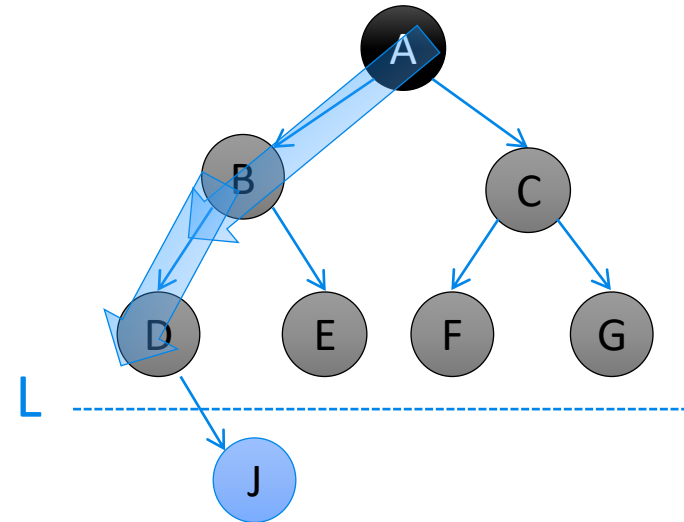
How Can We Make DFS Usable?

- Problem: Unbounded trees!
- Solution – **Depth-Limited Search**:
 - Identify a predetermined depth limit, L
 - Run DFS and ignore any successors beyond L



Limitations of DFS-Limited

- An incorrect choice of L dooms us!
- Question: how should we choose L?
 - Well, what do we want it to do?



Iterative Deepening Search (IDS)

- Our DFS problem:
 - We won't find a solution if $d > L$
- Our DFS solution:
 - Iteratively increase L
 - Stop when the shallowest goal node has been reached
- Analysis:
 - Complete if branching factor is finite
 - Optimal for problems where all actions have the same cost
 - Space complexity: $O(bd)$ – same as DFS for depth d !
 - Question: how wasteful is it to regenerate trees?

IDS

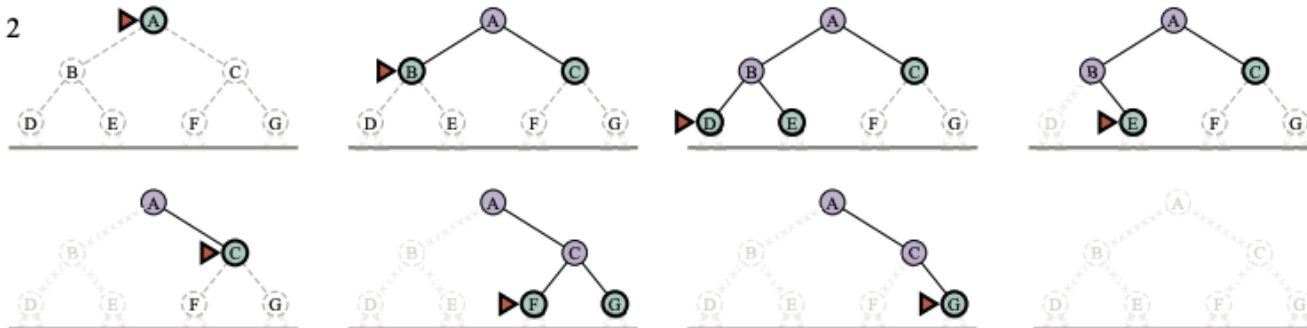
limit: 0



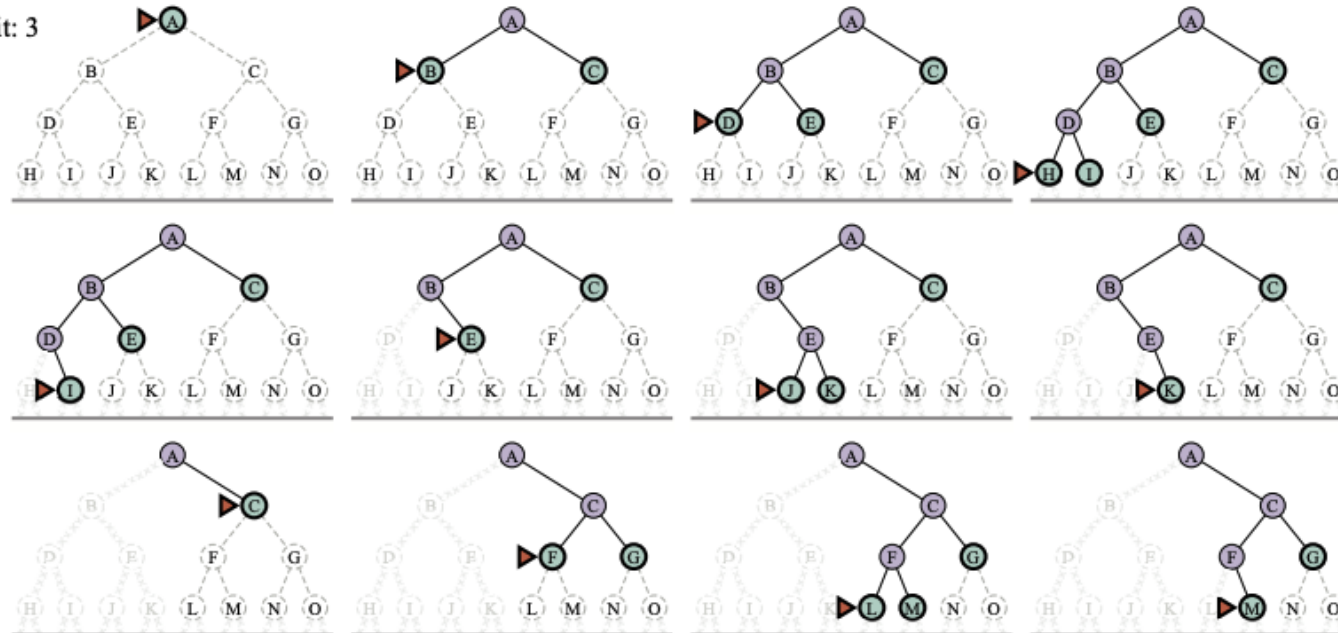
limit: 1



limit: 2



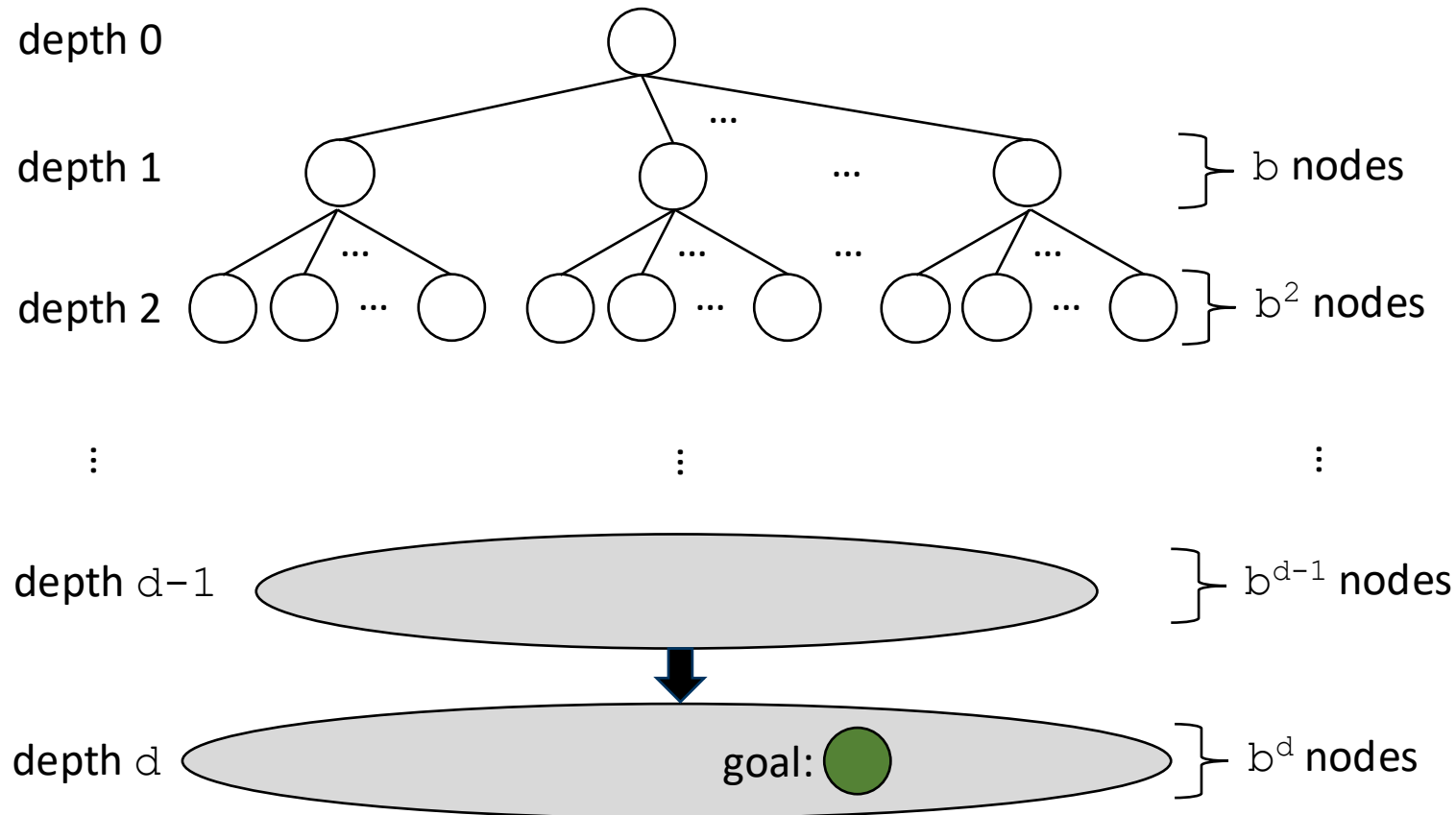
limit: 3



IDS: Time Complexity

- Proportionately, most nodes are at the bottom of the tree
- Node regeneration
 - Most nodes are at the bottom of the tree
 - Nodes at bottom are generated once $\rightarrow d$
 - Nodes at next lowest level are generated twice $\rightarrow 2(d-1)$
 - Nodes at the top are generated d times $\rightarrow d$
- Total number of generated nodes:
$$N(\text{IDS}) = (d) * b + (d-1) * b^2 + \dots + (1) b^d$$
- Time Complexity: $O(b^d)$  Asymptotically the same as BFS!

How many times does IDS generate each level?



times generated
d
d-1
⋮
2
1

Space complexity:
Same as DFS: $O(bd)$
If we stored nodes to save regeneration, space complexity would be like BFS!

Number of nodes generated (**time complexity**):

$$db + (d-1)b^2 + (d-2)b^3 + \dots + 2b^{d-1} + b^d = O(b^d)$$

Asymptotically the same as BFS!

Summary of Uniformed Search Methods

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹
Optimal?	Yes ³	Yes	No	No	Yes ³
Time Complexity	$O(b^d)$	$O(b^{l+(C^*/\epsilon)})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space Complexity	$O(b^d)$	$O(b^{l+(C^*/\epsilon)})$	$O(bm)$	$O(bl)$	$O(bd)$

↑
Exponential in Depth
↑
Linear in Depth

1. Complete if b is finite and the state space either has a solution or is finite
 2. Complete if all action costs are $\geq \epsilon > 0$
 3. Cost-optimal if action costs are identical
- These caveats are important!

Demo!

<https://clementmihailescu.github.io/Pathfinding-Visualizer/>

